the query does. That is, the results "must not match what's matched to the query in the clause". Hence, because the `terms` query is wrapped in a `must_not` clause, the result is the opposite: it fetches all products that are *not* manufactured by Samsung and Philips.

The problem with the query is that it fetches *all* products (TVs, fridges, monitors, and so on). But, remember, our requirement is to get *TVs only* that are *not* manufactured by Samsung or Philips. We should modify the query to fetch TVs, omitting those particular brands.

If you are thinking we should add a `must` clause to the `must_not` clause in listing 11.5, you are correct. We can create a `term` query wrapped in a `must` clause to fetch all TVs and then remove (filter out) the specific brands from the results using `must_not`. Let's update the query to reflect this.

Listing 11.6 Filtering out certain brands

```
GET products/_search
{
 "query": {
   "bool": {
     "must_not": [
       {
         "terms": {
           "brand.keyword": [
             "Philips",
             "Samsung"
           ]
         }
       }
     ],
     "must": [
       {
         "match": {
           "product": "TV"
         }
       }
     ]
   }
 }
}
```

The must_not clause that ignores a set of brands

A must clause that looks for (matches) TVs

We have two query clauses: a `must` and a `must_not`, both wrapped in a single `bool` compound query. Now we are searching for TVs that are not the two specific brands. Can we add more to the `must_not` clause? Yes; it can be enhanced with multiple leaf queries, as we discuss in the following section.

11.3.5 Enhancing the must_not clause

Similar to how we enhanced the query for `must` clauses (section 11.3.3), it's a no-brainer to provide multiple query criteria inside the `must_not` clause. For example, in addition to fetching products that are not manufactured by Philips and Samsung, we